



Credit Risk Production System

[image] [image] [image] [image] [image] [image] [image] [image] [image]
[image] [image]

Production-oriented credit-risk decisioning platform connecting ML, MLOps & LLMOps into one system.

ML produces the quantitative risk signal · Policy controls the business decision · LLM provides policy-grounded explanation

[Architecture](#) · [Performance](#) · [API](#) · [Roadmap](#)



Production Goal

Move credit-risk ML and LLM/RAG work from experimentation into a **production-oriented AI platform** across three engineering areas:

Area	Responsibility
ML Engineering	Build, package, evaluate & serve credit-risk models
MLOps	Model lifecycle, deployment, monitoring & automation
LLMOps	RAG, LLM deployment, evaluation, observability & integration

Core principle: The **ML layer** produces the quantitative prediction · The **policy layer** turns it into a business decision · The **RAG/LLM layer** provides policy-grounded explanation. The LLM never replaces the ML model as the source of the numerical risk score.



Architecture

```
flowchart TD
    A[👤 Customer Data] --> B[⚡ FastAPI Request]
    B --> C[🔧 Feature Processing]
    C --> D[📦 MLflow Model]
    D --> E[🎯 Prediction + Probability]
    E --> F[⚖️ Policy Engine]
    F --> |High score| G[✅ AUTO_APPROVE]
    F --> |Mid score| H[🔍 MANUAL_REVIEW]
    F --> |Low score| I[❌ AUTO_REJECT]
    H --> J[📖 RAG Retrieval · ChromaDB]
    J --> K[🧠 Hugging Face LLM]
    K --> L[📝 Explanation / Action]
    G & H & I & L --> M[🗄️ PostgreSQL · Audit]
    M --> N[📊 Prometheus]
    N --> O[📊 Grafana]
```

Tool responsibilities (click to expand)

Tool	Role
Python / Scikit-learn / XGBoost	ML development
MLflow	ML tracking & model registry
Kubeflow	ML pipeline orchestration
FastAPI	Production API
PostgreSQL 15 Alpine	Application & audit database
ChromaDB	RAG vector store
Hugging Face	LLM deployment / inference
Docker	Containerization
Kubernetes	Production orchestration
Prometheus	Metrics collection
Grafana	Monitoring & visualization

Model Performance

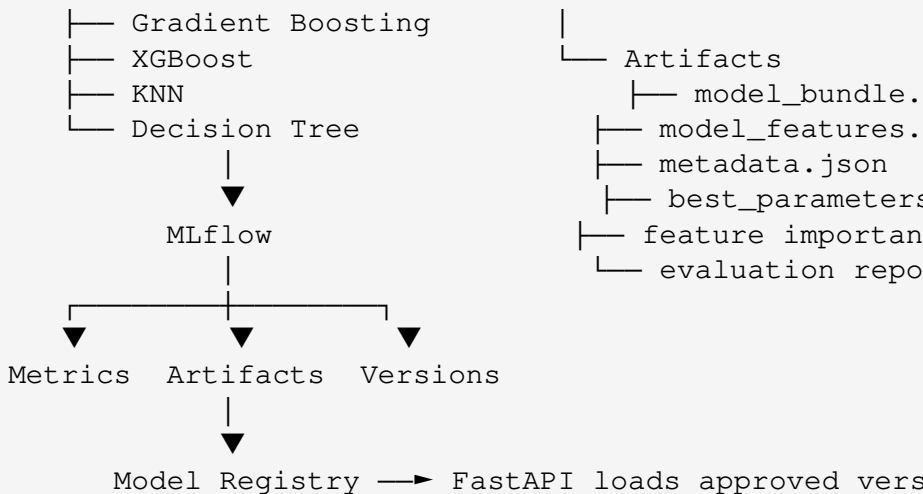
Note: The unusually high evaluation results must be validated for trustworthiness before production selection.

Rank	Model	Accuracy	F1 Score	ROC AUC	Status
	Gradient Boosting	99.53%	99.53%	99.99%	 Production candidate
	XGBoost	99.52%	99.52%	99.99%	 Registered
	Decision Tree	99.46%	99.46%	99.91%	 Registered
4	Random Forest	98.98%	98.97%	99.98%	 Registered
5	Logistic Regression	96.81%	96.72%	98.85%	 Registered
6	KNN	77.58%	75.58%	89.83%	 Registered

Production selection considers: reliability · inference latency · model stability · probability quality · business cost · explainability · operational complexity.

[↻ MLflow model lifecycle \(click to expand\)](#)

```
model_bundle.joblib
├── Logistic Regression
├── Random Forest
├── Accuracy / Precision
└── F1 Score / ROC AUC
```



API Endpoints

Endpoint	Description
GET /health	Liveness probe
GET /ready	Readiness probe (model + DB + LLM)
POST /api/v1/predict	Raw prediction + class probabilities
POST /api/v1/risk/score	Standardized risk score (0–100)
POST /api/v1/risk/decision	Score → policy decision (AUTO_APPROVE / MANUAL_REVIEW / AUTO_REJECT)
POST /api/v1/llm/explain	RAG-grounded explanation for review cases

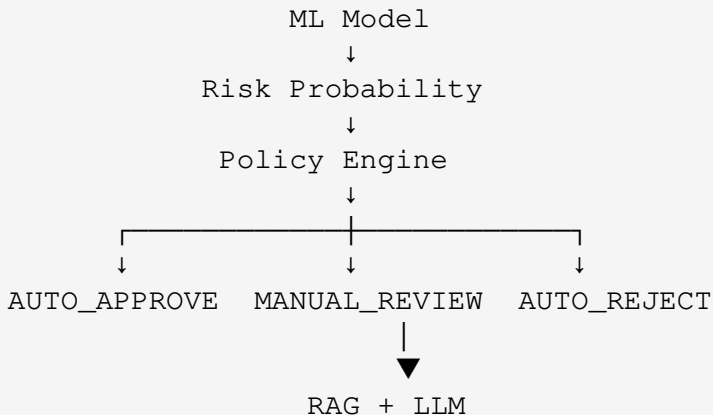
Example: decision + LLM response structure

```
{
```

```
"risk_level": "MEDIUM_HIGH",
"reason": [
  "Recent delinquency activity",
  "High credit utilization"
],
"recommended_action": "MANUAL_REVIEW",
"policy_references": [
  "Delinquency Classification"
]
}
```

The ML score remains the authoritative numerical prediction — the LLM response never becomes the source of the risk calculation.

⚖️ Decision Engine



This separation makes every decision **understandable and auditable**.

📈 Observability

Layer	Metrics
API	Request count · error rate · latency · HTTP status
ML	Prediction volume · model version · inference latency · prediction distribution
Decision	AUTO_APPROVE / MANUAL_REVIEW / AUTO_REJECT rates
LLM	Requests · latency · errors · token usage · fallback count

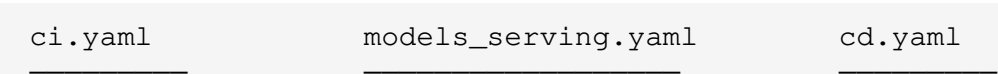
Production Services → Prometheus → Grafana → Engineering

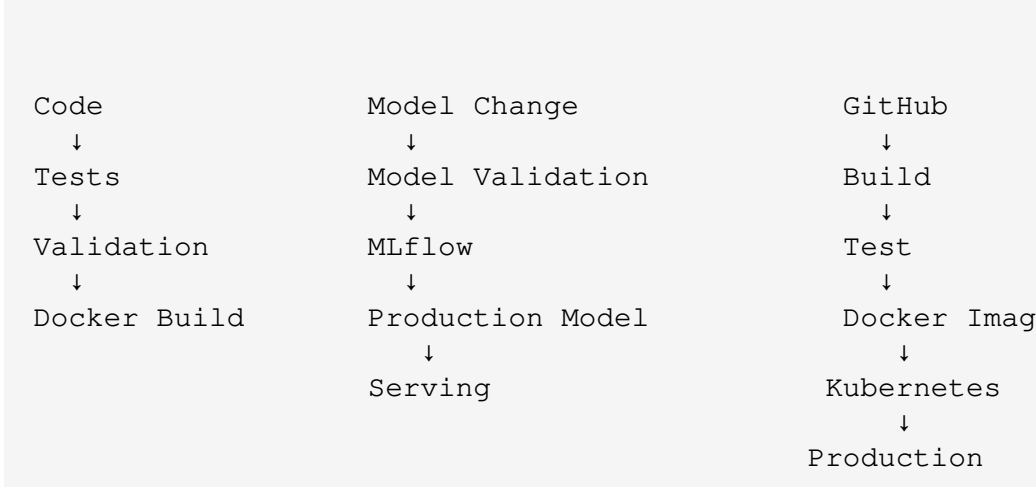


Development Roadmap

- ☒ **Phase 1 — ML Production** · Validate `model_bundle.joblib` · Register 6 models in MLflow · Select production candidate
- ☐ **Phase 2 — API** · FastAPI app · Prediction & decision endpoints · MLflow integration · PostgreSQL persistence
- ☐ **Phase 3 — Decision Engine** · Policy rules · `AUTO_APPROVE / AUTO_REJECT / MANUAL_REVIEW`
- ☐ **Phase 4 — LLMops** · ChromaDB + RAG · Hugging Face integration · Structured responses · Evaluation + interaction metadata
- ☐ **Phase 5 — Infrastructure** · Dockerize services · PostgreSQL 15 Alpine · Kubernetes deployment · Health checks
- ☐ **Phase 6 — Observability** · Prometheus metrics · Grafana dashboards · ML / decision / LLM monitoring
- ☐ **Phase 7 — CI/CD** · `ci.yaml` · `models_serving.yaml` · `cd.yaml` · Automated validation, builds & deployment

↻ GitHub Actions workflow structure





Configuration

Sensitive configuration is managed via environment variables and deployment secrets — never committed to Git.

```
HUGGINGFACE_API_KEY=...
MLFLOW_TRACKING_URI=...
DATABASE_URL=...
POSTGRES_PASSWORD=...
```

Project structure

```
credit_risk_production/
├── app/                                # FastAPI application
│   └── api/
├── database/
│   └── LLM/
│       ├── chroma_store/              # RAG vector store
│       └── outputs_llm/
│           ├── llm_decisions.csv
│           ├── model_artifacts/
│           │   └── model_bundle.joblib # 6-model bundle
│           └── pipeline_manifest.json
```

```
├── rag_config.json
└── .github/
    ├── ci.yaml
    ├── cd.yaml
    └── models_serving.yaml
```

Engineering Scope

ML Engineer

Feature engineering, training, evaluation, packaging, and making the approved model available for inference. Also responsible for validating whether the current unusually high evaluation results are trustworthy before production selection.

MLOps Engineer

Makes the ML system **reproducible · deployable · observable · versioned · maintainable** — MLflow registry, Kubeflow orchestration, Docker, Kubernetes, GitHub Actions, Prometheus/Grafana.

LLMOps Engineer

```
RAG → ChromaDB → LLM → Hugging Face → Evaluation → Monitor
```

Makes the LLM system **versioned · observable · reproducible · policy-grounded · safely integrated** with the ML decision system.

Production principle: ML produces the quantitative risk signal · the policy engine controls the business decision · the LLM provides policy-grounded explanation and assistance.